project

Week 1 homework

R05, python vs java:

Typing: Python variables can hold any type. Java variables have a fixed declared type, so String player_hp = 100; does not compile.

Compilation: Python finds errors only when a line runs. Java checks the whole file with javac first, so errors like a missing return are caught before the game starts.

Entry point: Python runs a script from top to bottom. Java always starts from public static void main(String[] args), and the name must be exactly main.

Example run:

Starting the game:

```
[nehorai@Nehorais-MacBook-Pro ~ % cd ~/Desktop/project_java_game/student-starter
 nehorai@Nehorais-MacBook-Pro student-starter % java -cp build edu.starter.Main

 press 1 to attack, press 2 to defend, press 3 to use your speical, press 4 to recover your resource
```

After every legal input, the enemy counterattacks.

Attacking (pressing 1):

```
press 1 to attack, press 2 to defend, press 3 to use your speical, press 4 to recover your resource
1
Player_hp90
Enemy_hp90
Turn_counter1
Player_resources5
```

Player and enemy hp went down by 10.

Using your special (pressing 3):

```
3
Player_hp80
Enemy_hp75
Turn_counter2
Player_resources2
```

Enemy hp went down by 15, player hp went down by 10.

Defending (pressing 2)

```
2
Player_hp75
Enemy_hp75
Turn_counter3
Player_resources2
```

Enemy hp didn't change, while player hp went down by only 5.

Recovering the player resource (pressing 4)

```
4
Player_hp65
Enemy_hp75
Turn_counter4
Player_resources5
```

Player hp went down by 10, while player resource went up by 3.

Using your special again (pressing 3):

```
3
Player_hp55
Enemy_hp60
Turn_counter5
Player_resources2
```

Player hp went down by 10 while enemy hp went down by 15.

Illegal input (pressing 5)

```
5
illegal input
Player_hp55
Enemy_hp60
Turn_counter5
Player_resources2
```

Nothing happened due to the input being illegal, every stat stayed the same.

Spamming attack until the game is over (both hp stats will go down by 10 each time):

```
1                 1                 1
Player_hp45       Player_hp35       Player_hp25
Enemy_hp50        Enemy_hp40        Enemy_hp30
Turn_counter6     Turn_counter7     Turn_counter8
Player_resources2 Player_resources2 Player_resources2
```

```
1                   1
Player_hp15         Player_hp5
Enemy_hp20          Enemy_hp10
Turn_counter9       Turn_counter10
Player_resources2 Player_resources2
```

Attacking one last time:

```
1
Player_hp5
Enemy_hp0
Turn_counter11
Player_resources2
player won, good job
```

Enemy hp went down to zero, so the player won. (Player hp didn't go down because a dead participant can't attack).

Test runs:

```
nehorai@Nehorais-MacBook-Pro ~ % cd ~/Desktop/project_java_game/student-starter

nehorai@Nehorais-MacBook-Pro student-starter % javac -d build src/main/java/edu/starter/*.java

nehorai@Nehorais-MacBook-Pro student-starter % java -cp build edu.starter.Main test
W01-ATK-01 passed
W01-DEF-01 passed
W01-SPC-01 passed
W01-RES-01 passed
EDGE-SPC passed
nehorai@Nehorais-MacBook-Pro student-starter %
```

All automated checks pass: W01-ATK-01, W01-DEF-01, W01-SPC-01, W01-RES-01 and boundary cases for Special at 3 and 2 resource. W01-INV-01 is shown manually in the example run above.

Known limitations:

If the input stream ends (pressing ctrl + d), the game keeps printing "illegal input" forever instead of exiting.

Design note:

The main design decision that I made was to store the game state in a small class called Things, instead of keeping it in local variables inside main.

At first, the player HP, enemy HP, resource and defense were local variables, and each action was a static method. This caused a major problem, a method receives a copy of an int, so it can't directly change variables that belong to main. Every change had to be returned and saved again. Special was the hardest case, because it changes two values at once (the resource and the enemy HP), while a method can return only one value.

I first tried returning an int[] array with both values. It worked, but it was easy to mix up which index meant what, and it made the code messier and more difficult to add stuff.

Moving the state into Things solved this. Each action is now a short method that updates the fields directly, so there is no need to return and reassign values. It also made testing easy, I can create any exact starting state, and check one action at a time.

The trade-off is that the workshop asked for local variables, and this is not exactly that. Also, one class currently holds both the player and the enemy state.

Ai use

I asked an AI assistant to look for logic errors in my game loop. It found a few things, and I checked two of them by running the game.

First one: it said Defend hits the player twice in the same turn. I pressed 2 at full HP and the player lost 15 HP instead of 5, so it was right. The problem was that defend() called the counterattack itself, and then the enemy attacked again at the end of the loop. I changed defend() so it only turns on pending_defense, and now the same input only takes 5 HP.

Second one: it said HP can go below zero. I pressed 1 nine times so the enemy was at 10 HP, then used Special, and the enemy ended up at -5. I added a check in attack() and counterattack() so HP stops at 0, and now the same input ends at 0.

Week 2 homework

Tests:

javac -d build src/main/java/edu/starter/*.java
java -cp build edu.starter.Main test
[Week 1] W01-ATK-01 passed
[Week 1] W01-DEF-01 passed
[Week 1] W01-SPC-01 passed
[Week 1] W01-RES-01 passed
[Week 1] EDGE-SPC-3 passed
[Week 2] T1 / W02-OBJ-01 passed
[Week 2] T2 / W02-ENC-01 passed
[Week 2] T3 / W02-ENC-02 passed
[Week 2] T4 / W02-ENC-03 passed
[Week 2] T5 / W01-INV-01 passed (counterattack part: manual)
[Week 2] EDGE-HP-5 passed

Run 1 – won

Only attacks. Enemy 1 reaches 0 HP and does not counterattack (player stays at 80). Encounter 2 starts with the same player at 80 HP against a new 45-HP enemy.

=== Encounter 1 ===
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 100 | Resource: 5 | Enemy HP: 30
1
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 90 | Resource: 5 | Enemy HP: 20
1
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 80 | Resource: 5 | Enemy HP: 10
1
You have killed the enemy
=== Encounter 2 ===
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 80 | Resource: 5 | Enemy HP: 45
1
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 70 | Resource: 5 | Enemy HP: 35
1
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 60 | Resource: 5 | Enemy HP: 25
1
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource

Player HP: 50 | Resource: 5 | Enemy HP: 15
1
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 40 | Resource: 5 | Enemy HP: 5
1
You have killed the enemy
you have won good job

Run 2 – lost

Only defend. The player loses 5 HP each turn and is defeated in encounter 1, so encounter 2 never starts. The invalid inputs (“22” and an empty line) change nothing and the enemy does not attack.

=== Encounter 1 ===
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 100 | Resource: 5 | Enemy HP: 30
2
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 95 | Resource: 5 | Enemy HP: 30
2
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 90 | Resource: 5 | Enemy HP: 30
2
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 85 | Resource: 5 | Enemy HP: 30
2
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 80 | Resource: 5 | Enemy HP: 30
2
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 75 | Resource: 5 | Enemy HP: 30
2
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 70 | Resource: 5 | Enemy HP: 30
2
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 65 | Resource: 5 | Enemy HP: 30
2
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 60 | Resource: 5 | Enemy HP: 30
2
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 55 | Resource: 5 | Enemy HP: 30
22
ilegal input please try again
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource

Player HP: 55 | Resource: 5 | Enemy HP: 30
2
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 50 | Resource: 5 | Enemy HP: 30

ilegal input please try again
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 50 | Resource: 5 | Enemy HP: 30
2
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 45 | Resource: 5 | Enemy HP: 30
2
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 40 | Resource: 5 | Enemy HP: 30
2
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 35 | Resource: 5 | Enemy HP: 30
2
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 30 | Resource: 5 | Enemy HP: 30
2
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 25 | Resource: 5 | Enemy HP: 30
2
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 20 | Resource: 5 | Enemy HP: 30
2
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 15 | Resource: 5 | Enemy HP: 30
2
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 10 | Resource: 5 | Enemy HP: 30
2
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 5 | Resource: 5 | Enemy HP: 30
2
You have been defeated!

Run 3 – special fails

Special works once (resource 5 → 2, enemy 30 → 15). The next two specials are rejected because there is only 2 resource, and the player HP does not change. Encounter 2 starts with 80 HP and 2 resource, so both values carried over.

=== Encounter 1 ===
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 100 | Resource: 5 | Enemy HP: 30

3
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 90 | Resource: 2 | Enemy HP: 15
3
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 90 | Resource: 2 | Enemy HP: 15
3
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 90 | Resource: 2 | Enemy HP: 15
1
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 80 | Resource: 2 | Enemy HP: 5
1
You have killed the enemy
=== Encounter 2 ===
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 80 | Resource: 2 | Enemy HP: 45
1
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 70 | Resource: 2 | Enemy HP: 35
1
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 60 | Resource: 2 | Enemy HP: 25
1
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 50 | Resource: 2 | Enemy HP: 15
1
press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource
Player HP: 40 | Resource: 2 | Enemy HP: 5
1
You have killed the enemy
you have won good job

Design note – week 2

The Combatant class is one fighter. It can be the player or an enemy, they are the same class but different objects. Combatant knows its own name, max hp, current hp, resource and if it has a pending defense. All the fields are private so only its own methods can change them. take_damage makes sure hp never goes under 0 and uses the defense if there is one, recover never goes over 10, and special only works with at least 3 resource.

playBattle is the battle coordinator. It reads the input, decides who gets hit and checks when the battle is over. It gets the player and the enemy from outside and returns true if the player won.

main creates the player only once and gives the same player object to both battles. Because it is the same object and not a copy, the hp and resource from battle 1 are still there in battle 2. The enemies are new objects, so enemy B always starts at 45 no matter what happened to enemy A. Between the battles main clears the defense.

One thing I did not put in Combatant is the 15 damage of the special. special() only pays the 3 resource and says if it worked, and playBattle gives the damage to the enemy. A combatant should only change itself and not know about other combatants.

AI note – week 2

A useful suggestion from the AI was to split special() so the Combatant only pays the 3 resource and playBattle gives the 15 damage to the enemy, because before that my special was hurting the player himself. A suggestion I did not take was to pass the encounter number as a parameter to playBattle; I used a static counter instead because it was simpler for me to call playBattle the same way every time. To check the code I used test T1

(two enemies with 30 and 45 HP, 10 damage only to the first one) and a manual run with 3, 3 ,3, 1, 1 that showed the player starts encounter 2 with 80 HP and 2 resource.

The AI also helped me write the automated tests and a first draft of the design note.

Week 3

Exit ticket workshop:

The HP cap is enforced inside the heal method in the Combatant class.

One reward is guaranteed because we call the reward code only one time inside the if of winning Encounter 1, and not inside a loop.

All the player's fields persist into the second fight except the temporary defense.

Traces:

First run- Recovery:

=== Encounter 1 ===

press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource

Player HP: 100 | Resource: 5 | Enemy HP: 30 | Attack: 10

3

press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource

Player HP: 90 | Resource: 2 | Enemy HP: 15 | Attack: 10

1

press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource

Player HP: 80 | Resource: 2 | Enemy HP: 5 | Attack: 10

1

You have killed the enemy

press 1 for Potion (+25 HP), press 2 for X Attack (+2 Attack)

1

=== Encounter 2 ===

press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource

Player HP: 100 | Resource: 2 | Enemy HP: 45 | Attack: 10

The player hp went up from 80, but not over the max of 100, therefore the recovery works.

Second run- Power:

=== Encounter 1 ===

press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource

Player HP: 100 | Resource: 5 | Enemy HP: 30 | Attack: 10

3

press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource

Player HP: 90 | Resource: 2 | Enemy HP: 15 | Attack: 10

1

press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource

Player HP: 80 | Resource: 2 | Enemy HP: 5 | Attack: 10

1

You have killed the enemy

press 1 for Potion (+25 HP), press 2 for X Attack (+2 Attack)

7

illegal input please try again

8

illegal input please try again

9

illegal input please try again

2

=== Encounter 2 ===

press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource

Player HP: 80 | Resource: 2 | Enemy HP: 45 | Attack: 12

1

press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource

Player HP: 70 | Resource: 2 | Enemy HP: 33 | Attack: 12

Invalid inputs (7, 8, 9) were rejected and the state didn't change. After choosing X Attack, Encounter 2 started with HP 80, Resource 2 and Attack 12, so Power only changes Attack.

Third run- No reward after defeat:

=== Encounter 1 ===

press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource

Player HP: 100 | Resource: 5 | Enemy HP: 30 | Attack: 10

2

press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource

Player HP: 95 | Resource: 5 | Enemy HP: 30 | Attack: 10

.

.

.

.

press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource

Player HP: 25 | Resource: 5 | Enemy HP: 30 | Attack: 10

2

press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource

Player HP: 20 | Resource: 5 | Enemy HP: 30 | Attack: 10

2

press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource

Player HP: 15 | Resource: 5 | Enemy HP: 30 | Attack: 10

2

press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource

Player HP: 10 | Resource: 5 | Enemy HP: 30 | Attack: 10

2

press 1 to attack, 2 to defend, 3 to use your special, 4 to recover your resource

Player HP: 5 | Resource: 5 | Enemy HP: 30 | Attack: 10

2

You have been defeated !

After the player is dead, we don’t get the reward option and encounter 2 didn’t start.

Tests:

[Week 1] W01-ATK-01 passed

[Week 1] W01-DEF-01 passed

[Week 1] W01-SPC-01 passed

[Week 1] W01-RES-01 passed

[Week 1] EDGE-SPC-3 passed

[Week 2] T1 / W02-OBJ-01 passed

[Week 2] T2 / W02-ENC-01 passed

[Week 2] T3 / W02-ENC-02 passed

[Week 2] T4 / W02-ENC-03 passed

[Week 2] T5 / W01-INV-01 passed (counterattack part: manual)

[Week 2] EDGE-HP-5 passed

[Week 3] W03-RWD-01 passed

[Week 3] EDGE-HEAL-CAP passed

[Week 3] W03-RWD-02 passed

[Week 3] T6 passed

[Week 3] W03-RWD-03/04/05, W03-END-01: manual (see Main.main)

Design note:

The reward is split between three parts of the program. Combatant owns the player's state, so it is the only class that changes HP and Attack. heal() and attackBoost() are private, and the only way to get a reward from outside is applyReward(). It checks rewardClaimed: the first call applies the reward and returns true, and any later call returns false and changes nothing. This prevents a second reward even if the method is called in the wrong order.

heal() enforces the HP cap: HP = min(maxHP, oldHP + 25), while resource and Attack stay the same. attackBoost() only adds 2 to the basic Attack. Special is a separate action with a fixed 15 damage that playBattle applies, so Power does not change it.

main is the coordinator. Only main knows if Encounter 1 was won, because playBattle returns true or false. That is why the precondition "reward only after a victory" is checked in main: applyReward is called only inside the if of winning Encounter 1. If the player loses, the game ends with no reward and no second battle.

The reward is an enum with two values, and its menu is separate from the battle menu. An invalid choice prints an error and asks again in a loop, without touching the player.

The invariants that always hold are 0 <= HP <= maxHP, 0 <= resource <= 10, and Attack never decreases.

Ai reflection:

A useful suggestion from the AI was to make heal() and attackBoost() private and add one method, applyReward(), with a rewardClaimed flag. Before that, main could call heal() as many times as it wanted, so the “one reward” rule was only kept by the order of the code in main. Now Combatant itself rejects a second reward.

A suggestion I rejected came from Copilot while I was writing the contracts. It auto-completed some of the @pre/@post comments, but a few of them didn’t match what the code actually does. For example, one compared values after the method had already changed them, and another put a @post on a class instead of a method. I checked each suggestion against the code and rewrote the wrong ones myself. I also didn’t take the suggestion to move every class (Combatant and the reward enum) into its own file. I only moved the tests into Tests.java, because that was the change I needed this week.

To check the code myself, I ran Trace 2 by hand. I typed 7, 8 and 9 as invalid reward choices, then chose X Attack. Encounter 2 started with HP 80, Resource 2 and Attack 12, and the first attack took the enemy from 45 to 33, so Power really changed the basic Attack. I also ran the automated tests, and all 15 passed, including T6 (a second reward is rejected). The AI also helped me write the contracts, the Week 3 tests and a first draft of this design note.

Acceptance test table:

ID	Setup / action	Expected outcome	What happened
T1	Start from HP 90; choose Recovery	HP 100; resource/Attack unchanged	HP went to 100, resource and attack stayed the same. Pass (test W03-RWD-01, Trace 1)
T2	Start from HP 100; choose Recovery	HP stays 100; reward consumed	HP stayed 100. Pass (test EDGE-HEAL-CAP)
T3	Start from Attack 10; choose Power	Basic Attack 12; Special still 15	Attack went to 12, special still does 15. Pass (test W03-RWD-02, Trace 2)
T4	After fight 1, enter invalid choice then Power	State unchanged on error; Power once	7, 8, 9 were rejected and nothing changed, then Power was applied once. Pass (Trace 2)
T5	Try to claim a reward before first victory	Rejected; no change	Can't happen: applyReward is only called after winning Encounter 1
T6	Try to select another reward after choosing one	Rejected; no change	Second applyReward returned false, nothing changed. Pass (test T6)
T7	Lose Encounter 1	No reward and no second battle	"You have been defeated!", no reward and no Encounter 2. Pass (Trace 3)
T8	Finish fight 1 at 80 HP and 2 resource	Those values persist unless reward changes HP	Encounter 2 started with HP 80 and resource 2 (Power), or HP 100 and resource 2 (Recovery). Pass (Traces 1 and 2)

T9	Defeat Enemy 2 with a finishing attack	Adventure won; no counterattack afterward	The enemy died and didn't attack back, "you have won good job". Pass (Week 2 Run 1)